

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности

Направление подготовки	02.03.01	Математика и компьютерные науки
Направленность подготовки бакалавриата	02.03.01_01	Системы искусственного интеллекта и суперкомпьютерные технологии
Квалификация		Бакалавр
Институт		Компьютерных наук и кибербезопасности
Кафедра		Высшая школа технологий искусственного интеллекта
Дисциплина		Генетические алгоритмы
Курс	4	Группа 5130201/20101

**ПОИСК ГИПЕРПАРАМЕТРОВ И ВЫБОР МОДЕЛИ СЕГМЕНТАТОРА ДЛЯ
ЗАДАЧИ ВЫДЕЛЕНИЯ ЗОНЫ ИНТЕРЕСА ТРАМВАЙНЫХ ПУТЕЙ**

Студент: _____

Салимли Айзек Мухтар Оглы

Преподаватель: _____

Большаков Александр Афанасьевич

«_____» _____ 20__ г.

ЗАДАНИЕ

1. Тема работы: «Распознавание посторонних объектов на трамвайных рельсах в режиме реального времени.»
 - 1.1. Скобцов, Ю. А. Эволюционные вычисления: Учебное пособие / Ю. А. Скобцов, Д. В. Сперанский. – М.: Национальный Открытый Университет «ИНТУИТ», 2012. – 331с., ил. – (Серия «Основы информационных технологий»).
 - 1.2. Алиев М. В., Бербенцев Д. А., Немыкин В. О., Алиева С. М. Алгоритмы распознавания объектов в видеопотоке и определение свойств их взаимного расположения // Вестник Адыгейского государственного университета. Сер. Естественно-математические и технические науки. – 2024. – № 2(341). – С. 27–34. DOI: 10.53598/2410-3225-2024-2-341-27-34.
 - 1.3. Зотов С. С., Яковлев А. А., Колчищев Д. А. Обнаружение объектов в реальном времени с помощью алгоритмов распознавания YOLO // Синергия наук. – 2018. – № 23. – С. 1245–1261.
 - 1.4. Чочиа П. А. Сегментация изображений на основе анализа расстояний в пространстве признаков // Информационные процессы. – 2020. – № 6. – С. 97–110.
 - 1.5. Zhang H., Ullah N, Sharnah A. CA-SegFormer: An improved coordinate attention-enhanced model for crack segmentation in concrete structures // Case Studies in Construction Materials. – 2025. – Vol. 23.
 - 1.6. Yanyan Z., Zhiliang S. Adioc loss: An Auxiliary descent IoC loss function // Engineering Applications of Artificial Intelligence - 2022. – №105453. – с. 117–133. DOI: <https://doi.org/10.1016/j.engappai.2022.105453>.
 - 1.7. YOLOv8 Documentation [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolov8/> (Дата обращения: 07.02.2026).
 - 1.8. YOLOv5 Documentation [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolov5/> (Дата обращения: 07.02.2026).
 - 1.9. Xie E., Wang W., Yu Z., Anandkumar A., Alvarez J. M., Luo P. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers // arXiv [cs.CV]. – 2021. – 28 Oct.
 - 1.10. RailSafeNet. train_SegFormer.py [Электронный ресурс] // GitHub. – Доступ: https://github.com/oValach/RailSafeNet/blob/main/train_SegFormer.py (Дата обращения: 14.03.2026).
 - 1.11. Мороз Е. С., Пикалёв Я. С. Анализ SOTA-архитектур для семантической аугментации // Информационные технологии и системы. – 2025. – № 4. – С. 39–60. DOI: 10.24412/2413-7383-2025-4-39-60-70.
 - 1.12. Xie E., Wang W., Yu Z., Anandkumar A., Alvarez J. M., Luo P. Details of MiT Series // arXiv [cs.CV]. – 2021. – (Supplementary material).
 - 1.13. DeepLabV3Plus-RailSem19 [Электронный ресурс] // GitHub. – Доступ: <https://github.com/pideyi1025/DeepLabV3Plus-RailSem19/tree/main> (Дата обращения: 14.03.2026).
 - 1.14. Springer Link. Статья по компьютерному зрению [Электронный ресурс] // Springer. – Доступ: <https://link.springer.com/article/10.1007/s11554-023-01380-x> (Дата обращения: 14.03.2026).

- 1.15. Instance Segmentation. Val [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/tasks/segment/#val> (Дата обращения: 14.03.2026).
 - 1.16. YOLOv8. Key Features [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolov8/#key-features-of-yolov8> (Дата обращения: 14.03.2026).
 - 1.17. YOLOv11. Key Features [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolov11/#key-features-of-yolov11> (Дата обращения: 14.03.2026).
 - 1.18. YOLO26 Documentation [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolo26/> (Дата обращения: 14.03.2026).
 - 1.19. Hyperparameter Tuning [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/guides/hyperparameter-tuning/> (Дата обращения: 14.03.2026).
2. Содержание работы
 - 2.1. Введение
 - 2.2. Актуальность темы
 - 2.3. Постановка задачи
 - 2.4. Компьютерное зрение и задача сегментации
 - 2.5. Сегментатор SegFormer
 - 2.6. Сегментатор DeepLabV3+ ResNet101
 - 2.7. Сегментатор YOLO-seg
 - 2.8. Подбор гиперпараметров методом эволюционной стратегии
 - 2.9. Выбор метода минимизации функции потерь
 - 2.10. Обучение сегментатора
 - 2.11. Анализ результатов
 - 2.12. Заключение
 3. Дата выдачи задания «6»февраля 2026 г.

Преподаватель: _____

Большаков А.А.

Задание принято к исполнению «6»февраля 2026 г.

Студент: _____

Салимли А.М.

Содержание

Введение	5
1 Актуальность темы	6
2 Постановка задачи	7
3 Компьютерное зрение и задача сегментации	8
4 Сегментатор SegFormer	10
5 Сегментатор DeepLabV3+ ResNet101	12
5.1 Архитектура ResNet101	12
5.2 Архитектура DeepLabV3+ ResNet101	12
6 Сегментатор YOLO-seg	14
7 Подготовка датасета	16
8 Стандартное обучение сегментаторов	17
8.1 Генетический алгоритм tune	17
8.2 Результаты подбора гиперпараметров	18
8.3 Выбор метода оптимизации	20
8.4 Эволюционный алгоритм CMA-ES	21
8.5 SGD	23
8.6 AdamW	23
9 Обучение сегментаторов	24
10 Результаты обучения	27
10.1 Результаты обучения сегментатора YOLOv8n-seg	27
10.2 Результаты обучения сегментатора YOLOv8s-seg	28
10.3 Результаты обучения сегментатора YOLOv11s-seg	29
10.4 Результаты обучения сегментатора YOLO26s-seg	30
10.5 Результаты обучения SegFormer B0	31
10.6 DeepLabV3+ ResNet101	32
11 Анализ результатов	33
Заключение	37
Список литературы	38

Введение

Обнаружение посторонних объектов на трамвайных путях - практическая задача для систем компьютерного зрения. В данной работе предлагается следующий подход: на первом этапе передачи видеопотока выполняется сегментация рельсов с помощью исследуемых моделей (SegFormer, DeepLabV3+ ResNet101, YOLOv8n-seg, YOLOv8s-seg, YOLOv11s-seg, YOLO26s-seg); на втором этапе с помощью библиотеки `opencv` дорисовываются ограничения опасной зоны которая является зоной рельс и прилегающей к ним области по ширине трамвая; далее на третьем этапе в выделенной опасной зоне применяется детектор YOLOv8 в режиме инференса для распознавания посторонних объектов в реальном времени (**второй и третий этапы не входит в курсовую работу**).

Обучается не модель детекции (YOLOv8 используется только на этапе инференса), а сегментатор. Для выбора модели сегментатора, рассматриваются три модели с разными архитектурами, в последствие сравнения, выбирается сегментатор удовлетворяющий критериям реального времени $30FPS$ и точности не менее 70%. Для таких задач требуется правильный подбор гиперпараметров и оптимизация функции потерь сегментатора. Стандартным промышленным подходом для минимизации функции потерь являются методы градиентного спуска, в частности стохастический градиентный спуск (SGD) и его адаптивные варианты. В рамках данной курсовой работы рассматривается генетический алгоритм основанный на эволюционной стратегии, для обеспечения автоматического подбора гиперпараметров и выбор алгоритма минимизации функции потерь сегментатора с помощью генетических алгоритмов и методов градиентного спуска.

Результатом работы является вывод оптимальных гиперпараметров для обучения сегментатора, выбор модели сегментатора и алгоритм минимизации функции потерь сегментатора.

В качестве генетических алгоритмов для минимизации функции потерь рассматриваются:

- Эволюционная стратегия с ковариационной матрицей из библиотеки **torch** (CMA-ES);

В качестве градиентных методов для минимизации функции потерь рассматриваются:

- Стохастический градиентный спуск (SGD);
- Адаптивный метод с разделением затухания весов из библиотеки **torch** (AdamW).

Для подбора гиперпараметров используется метод эволюционной стратегии, работающий по принципу отбора лучших особей (наборов гиперпараметров) по целевой метрике (fitness) и генерации новых кандидатов путём мутации.

В библиотеке Ultralytics метод `tune()` реализует подбор гиперпараметров на основе генетического алгоритма: задаётся начальный набор гиперпараметров и допустимая область поиска; на каждой итерации к текущему набору применяется оператор мутации (`_mutate`), получаемые варианты используются для обучения модели, после чего по метрикам (AP50, F1-score и др.) вычисляется fitness; в следующих итерациях эволюция опирается на наиболее успешные конфигурации. Кроссовер в базовой реализации не используется - поиск ведётся преимущественно за счёт мутации. Процесс повторяется до достижения заданного числа итераций или требуемого качества; результаты сохраняются (в том числе в CSV), оптимальные гиперпараметры выводятся в отдельный файл.

1 Актуальность темы

Планируемое открытие новых высокоскоростных трамвайных линий в Санкт-Петербурге повышает требования к надежности и скорости реакции систем мониторинга. Существующие методы визуального контроля, основанные на наблюдении персонала, иногда не обеспечивают необходимой полноты, непрерывности и быстродействия, особенно в условиях интенсивного городского трафика, плохой видимости или сложной погодной обстановки.

Следовательно, создание автоматизированной системы, способной в режиме реального времени с высокой точностью детектировать посторонние объекты на путях, является актуальной научно-прикладной задачей. Ее решение должно позволить, минимизировать риски аварийных ситуаций и повысить общую надежность и безопасность работы городского электротранспорта. Данное исследование посвящено сравнительному анализу методов оптимизации для минимизации функции потерь (градиентный спуск и генетические алгоритмы), выбор метода подбора гиперпараметров (генетический алгоритм) и конечный выбор модели сегментатора и направлено на поиск наиболее эффективного технического решения.

Такая система, может служить как вспомогательная система для водителя трамвая или основой для создания системы беспилотных трамваев.

2 Постановка задачи

В данной курсовой работе необходимо:

1. Найти или подготовить размеченный датасет изображений трамвайных путей;
2. Сформировать критерий реального времени для полного прогона конвейера (сегментация кадра, дорисовка опасной зоны, детекция объектов);
3. Разбить датасет на части обучения, валидации и тестирования;
4. Изучить скорости и точность инференса сегментаторов;
5. На основе стандартного обучения сегментатора, выбрать оптимальную модель;
6. Реализовать подбор гиперпараметров методом эволюционной стратегии;
7. Выбрать метод минимизации функции потерь;
8. Обучить сегментатор на выбранной модели и методе минимизации функции потерь;
9. Провести анализ результатов.

3 Компьютерное зрение и задача сегментации

Компьютерное зрение (Computer Vision) - это научная область искусственного интеллекта, целью которой является создание моделей и алгоритмов для автоматического извлечения, анализа и понимания полезной информации из цифровых изображений или видеопотоков. Формально, это задача преобразования данных из исходного пространства высокомерных пикселей (матрицы интенсивностей), в компактное пространство семантических описаний, пригодных для принятия решений. Формально можно задать отображение:

$$I(h, w) \rightarrow T_r,$$

где h - высота изображения, w - ширина изображения, а T_r это пространство семантических описаний, пригодных для принятия решения.

В случае поставленной задачи, T_r - будет являться разделенное изображение на семантически значимые области или экземпляры объектов.

Описанная задача, называется сегментацией изображения.

Подход сегментации можно рассмотреть как разбиение изображения на однородные области на основе анализа расстояний в пространстве признаков.

Можно более точно задать теоретико-множественную модель как отображение множества точек изображения в конечный набор значений (меток сегментов):

$$F : Img \rightarrow 1, 2, \dots, Q,$$

где Q - итоговое число сегментов. Критерием однородности служит оценка близости точек в объединенном пространстве признаков.

Пространство можно описать так:

Множество точек изображения $I(h, w)$, на дискретной решетке:

$$\Omega = (h, w) | h = 1, \dots, H, w = 1, \dots, W$$

Каждой точке, сопоставлен вектор интенсивности RGB:

$$x(h, w) \in \mathbb{R}^3$$

Тогда исходное пространство:

$$I_s = x(h, w) | ((h, w) \in I) \subset \mathbb{R}^3$$

Помимо цвета RGB, точке может быть присвоены иные признаки (в SegFormer эти признаки делятся на 4 уровня), например - контекстовый эмбединг вектора, семантические признаки.

Это пространство признаков $F \subset \mathbb{R}^d$

Тогда получаем отображение множества исходного пространства в множество пространства признаков.

$$Prepare : I_s \rightarrow F$$

То есть для каждого пикселя получая вектор признаков:

$$\vec{v}(h, w) = \text{Prepare}(x(h, w)) \in \mathbb{R}^d$$

За отображение точек в признаки, отвечает Encoder, после того как признаки были присвоены всем пикселям в изображение, они поступают на вход в Decoder, который отвечает за классификацию через слои нейронной сети получая на выходе тензоры, проходящие через *Argmax*, в результате чего идет преобразование тензора в пиксель. Далее к каждому декодированному пикселю применяется функция *Softmax*, которая присваивает вероятности принадлежности пикселя к классу и дорисовывается маска сегментации.

4 Сегментатор SegFormer

Сегментация формулируется как задача попиксельной классификации, решаемая нейронной сетью.

Математически такая сегментация описывается как присвоение каждому пикселю входного изображения метки класса. SegFormer - представляет собой структуру энкодер-декодера работающий по следующему алгоритму:

- Иерархический трансформер-энкодер:
 - Входное изображение $H \times W \times 3$ разбивается на батчи
 - Энкодер генерирует многоуровневые признаки F_i , с разрешением $\frac{H}{4}, \frac{H}{8}, \frac{H}{16}, \frac{H}{32}$
 - Далее механизм внимания, который снижает вычислительную сложность за счет уменьшения размерностей ключей K :

$$Attention(Q, K, V) = Softmax(\frac{QK^T}{\sqrt{d_{head}}})V$$

$$\hat{K} = ReshapeK, K = Linear(\hat{K})$$

- Координатное внимание:
 - Координатное внимание кодирует входное изображение по вертикали z_n^h и по горизонтали z_n^w
 - Далее кодируются точки по диагонали по главной z_n^m и второстепенной z_n^v

$$z_n^h(h) = \frac{1}{W} \sum_{i=0}^{\min H, W} x_n(h, i)$$

$$z_n^w(w) = \frac{1}{H} \sum_{j=0}^{\min H, W} x_n(j, w)$$

$$z_n^m(k) = \frac{1}{\min H, W} \sum_{k=0}^{\min H, W} x_n(k, k)$$

$$z_n^v(l) = \frac{1}{\min H, W} \sum_{l=0}^{\min H, W} x_n(l, H - 1 - l)$$

- Эти карты признаков конкатинируются и обрабатываются свертками 1×1 , для получения тензора внимания f , который затем разделяется обратно на 4 направления.
- Финальные веса g применяются к исходным признакам:

$$y_n(i, j) = x_n(i, j) \cdot g_n^h(i) \cdot g_n^w(j) \cdot g_n^m(k) \cdot g_n^v(l)$$

- После кодирования применяется легковесный MLP-декодер:
 - Признаки F_i от энкодера нормализуются и приводятся к единой размерности каналов C : $\hat{F}_i = Linear(C_i, C)(F_i)$
 - Все $\hat{F}_i$ апсемпляются до разрешения $\frac{H}{4} \times \frac{W}{4}$

- После признаки конкатенируются и пропускаются через MLP-слой для слияния:

$$F = \text{Linear}(4C, C)(\text{Concat}(\hat{F}_i))$$

- Финальный слой MLP предсказывает маску сегментации M размерности $\frac{H}{4} \times \frac{W}{4} \times N_{cls}$, где N_{cls} - число классов.

Для оценки качества сегментации используются метрики Precision и Recall которые вычисляются на основе пикселей матрицы ошибок (TP, FP, TN, FN).

Precision - отвечает за то, сколько доля правильных объектов среди всех, посчитала нейросеть.

Recall - отвечает за то, сколько всего найденных объектов среди всех реальных на картинке.

Целевой функцией для минимизации, является функция потерь Sparse Crosss-Entropy Loss, которая минимизирует разницу между предсказанным распределением вероятности для каждого пикселя и истинной маской:

$$L = - \sum_{c=1}^M y_c \log(\hat{y}_c),$$

где M - количество классов, y - бинарный индикатор принадлежности пикселя к классу C , $\hat{y}$ - предсказанная вероятность принадлежности к классу C .

Следовательно поставленная задача курсовой работы:

$$\theta^* = \text{argmin}_{\theta} L(f(M_p; \theta), M_t)$$

То есть ищем параметр θ ведущий к минимальной ошибке, предсказания для входных данных M_p , по сравнению с истинными M_t .

Ниже на рисунке 1 представлена архитектура SegFormer:

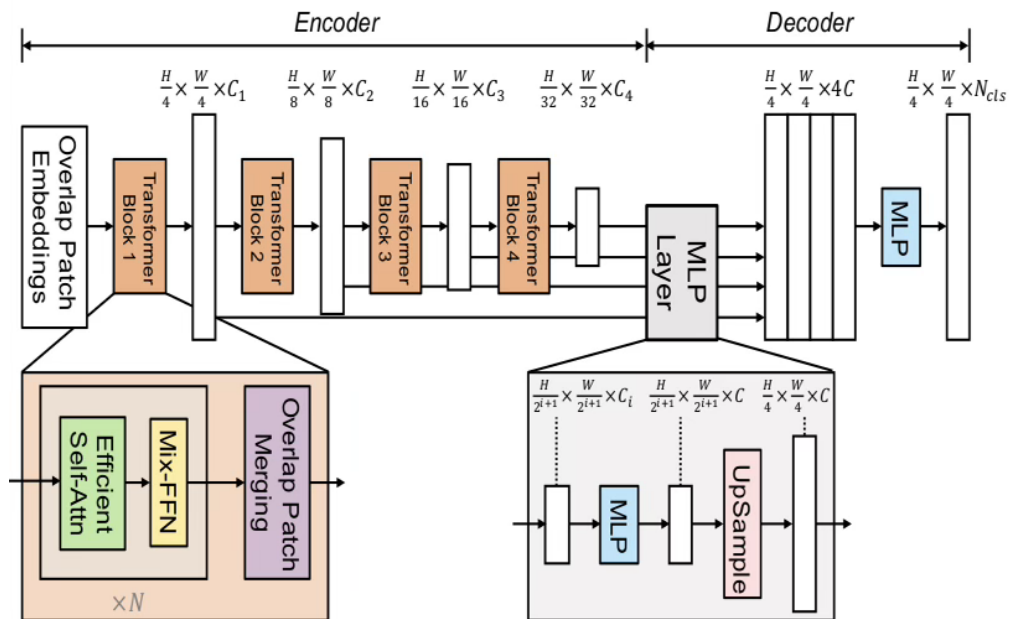


Рис. 1: Архитектура SegFormer

5 Сегментатор DeepLabV3+ ResNet101

5.1 Архитектура ResNet101

Энкодер DeepLabV3+ построен на основе сети ResNet-101, глубокой свёрточной сети с остаточными связями (residual connections). Основная идея ResNet - обучение остаточного отображения вместо прямого: если целевое отображение для слоя есть $H(\mathbf{x})$, блок обучает разность $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$, а выход формируется как $\mathbf{x}_{l+1} = f(\mathbf{x}_l + F(\mathbf{x}_l, \mathcal{W}_l))$, где f - функция активации (ReLU), $\mathcal{W}_l$ - параметры блока. Такая параметризация облегчает обучение очень глубоких сетей и устраняет проблему затухающего градиента.

В ResNet-101 используются *bottleneck*-блоки: остаточная функция F реализована тремя свёртками 1×1 , 3×3 , 1×1 . Обозначив вход блока $\mathbf{x}$, промежуточные выходы после batch normalization и ReLU имеем:

$$\mathbf{z}_1 = \sigma(\text{BN}(\mathcal{W}_1 * \mathbf{x})), \quad \mathbf{z}_2 = \sigma(\text{BN}(\mathcal{W}_2 * \mathbf{z}_1)), \quad F(\mathbf{x}) = \mathcal{W}_3 * \mathbf{z}_2, \quad (1)$$

где $*$ - свёртка, σ - ReLU. Выход блока: $\mathbf{x}_{l+1} = \sigma(\mathbf{x}_l + F(\mathbf{x}_l))$ при совпадении размерностей; при смене разрешения или числа каналов вход проецируется свёрткой 1×1 со stride: $\mathbf{x}_{l+1} = \sigma(\text{Proj}(\mathbf{x}_l) + F(\mathbf{x}_l))$.

Сеть ResNet-101 состоит из начального слоя (свёртка 7×7 , max pooling) и трех стадий (conv2_x - conv5_x) с числом блоков 3, 4, 23 соответственно. На первой подстадии каждой стадии пространственное разрешение при необходимости уменьшается в 2 раза (в первой свёртке 3×3 или в проекции), число каналов увеличивается (64, 128, 256, 512). В DeepLabV3+ последние блоки (например, в conv4_x и conv5_x) могут использовать атрозные свертки для сохранения плотности карт признаков.

5.2 Архитектура DeepLabV3+ ResNet101

Модель DeepLabV3+ с энкодером ResNet-101 объединяет энкодер с модулем ASPP (Atrous Spatial Pyramid Pooling) и декодер для восстановления границ объектов. Главное отличие от обычной свёрточной сети - использование атрозных (разрежённых) свёрток, позволяющих управлять разрешением карт признаков без потери охвата по полю обзора.

Для одномерного сигнала $x[i]$ атрозная свёртка с фильтром $\omega[k]$ длины K и скоростью расширения (dilation rate) r определяется как

$$y[i] = \sum_{k=1}^K x[i + r \cdot k] \cdot \omega[k]. \quad (2)$$

При $r = 1$ получается обычная свёртка; при $r > 1$ выборка входного сигнала разрежается, что эквивалентно увеличению эффективного поля обзора без роста числа параметров. Для двумерных карт признаков формула обобщается покомпонентно по пространственным индексам. В ResNet-101 при использовании в качестве энкодера DeepLabV3+ атрозные свёртки применяются в последних остаточных блоках для извлечения признаков с заданным output stride (например, 16 или 8).

Модуль ASPP захватывает контекст на нескольких масштабах. Он применяет к выходному тензору backbone несколько параллельных ветвей: свёртку 1×1 ; три атрозные свёртки 3×3 с разными скоростями расширения $r \in \{6, 12, 18\}$ (при output stride 16); и ветвь с глобальным усреднением по пространственным координатам (Image-level pooling) для глобального контекста. Выход ASPP - конкатенация результатов всех ветвей, пропущенная через свёртку 1×1 для приведения к 256 каналам:

$$Y_{\text{ASPP}} = \text{Conv}_{1 \times 1}([f_{1 \times 1}(X) \| f_{r=6}(X) \| f_{r=12}(X) \| f_{r=18}(X) \| f_{\text{pool}}(X)]), \quad (3)$$

где $\parallel$ обозначает конкатенацию по каналам, f_{pool} - глобальное усреднение с последующей свёрткой 1×1 .

В декодере признаки из ASPP сначала апсемплируются билинейной интерполяцией в 4 раза, затем конкатенируются с низкоуровневыми признаками энкодера (например, выход conv2_x в ResNet-101), предварительно приведёнными к меньшему числу каналов свёрткой 1×1 . После конкатенации применяются две свёртки 3×3 с 256 фильтрами для уточнения карт признаков, затем финальный билинейный апсемплинг в 4 раза восстанавливает исходное пространственное разрешение изображения.

В качестве функции потерь используется кросс-энтропия по пикселям:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c}, \quad (4)$$

где N - число пикселей, C - число классов, $y_{i,c}$ - истинная метка (one-hot по классам), $\hat{y}_{i,c}$ - предсказание модели (вероятность класса c в пикселе i).

Ниже на рисунке 2 представлена архитектура DeepLabV3+ ResNet101:

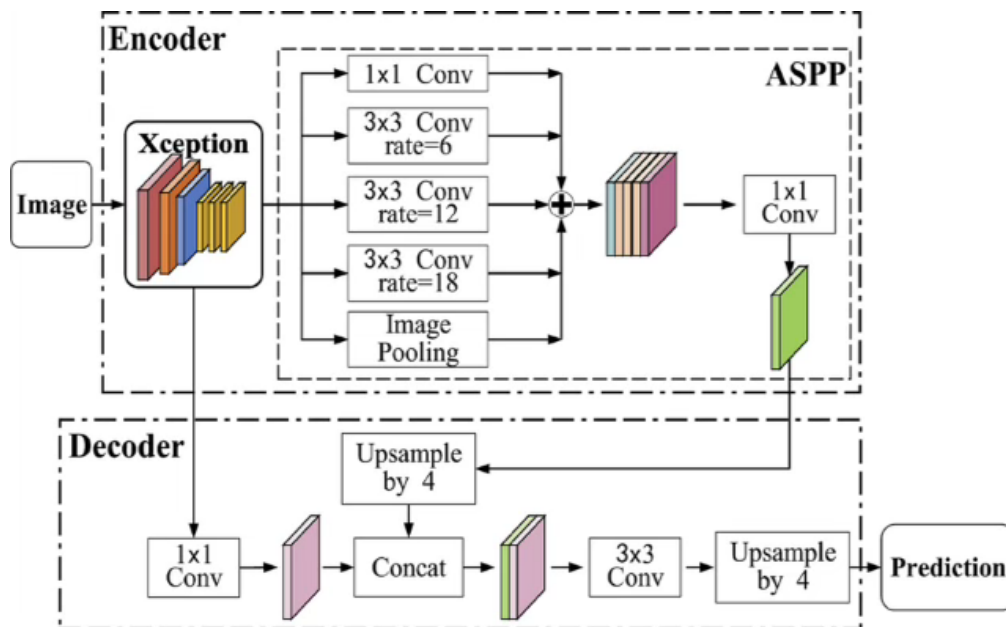


Рис. 2: Архитектура DeepLabV3+

6 Сегментатор YOLO-seg

Модель YOLO-seg выполняет попиксельную сегментацию: для каждого обнаруженного объекта предсказывается ограничивающий прямоугольник, класс, и пиксельная маска контура. Архитектура совмещает общую часть детектора (backbone, neck, head детекции) с модулем генерации масок по схеме прототипных масок, унаследованной от YOLACT.

Входное изображение проходит через backbone с блоками C3k2 (Cross Stage Partial с ядром 2): разбиение входа на две ветви, одна сохраняется, вторая обрабатывается параллельными свёртками с последующей конкатенацией для эффективного извлечения признаков. Модуль SPPF (Spatial Pyramid Pooling Fast) агрегирует контекст на нескольких масштабах через последовательный max pooling и свёртку. В neck используется блок C2PSA (CSP с Pyramid Squeeze Attention): пространственное внимание с ядрами разного размера и взвешивание каналов. На выходе neck формируется пирамида признаков для head-ов детекции и сегментации.

Детекция выполняется в безякорном формате: для каждой ячейки/позиции предсказываются смещения центра объекта (t_x, t_y) , размеры (t_w, t_h) , оценка объектности и распределение по классам. После декодирования координат и пороговой фильтрации получают множество детекций $\mathcal{D} = \{(b_i, c_i, s_i)\}$, где b_i - бокс, c_i - класс, s_i - уверенность.

Сегментация добавляет к каждой детекции маску. Модель выдаёт два типа выходов:

- **Прототипные маски $\mathbf{P}$:** тензор размерности $n_m \times H_p \times W_p$ ($n_m = 32$ в стандартной конфигурации), получаемый отдельной свёрточной ветвью (ProtoNet) от карт признаков neck. Каждый канал $P_k(h, w)$ интерпретируется как базисная пространственная карта.
- **Коэффициенты масок:** для каждой детекции i head предсказывает вектор $\mathbf{a}_i = (a_{i1}, \dots, a_{i, n_m})^\top \in \mathbb{R}^{n_m}$ (дополнительные каналы в выходе head детекции).

Итоговая маска для детекции i формируется как линейная комбинация прототипов с последующей поэлементной сигмойдой:

$$\tilde{M}_i(h, w) = \sigma \left(\sum_{k=1}^{n_m} a_{ik} \cdot P_k(h, w) \right), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (5)$$

где (h, w) - координаты в сетке прототипов размера $H_p \times W_p$. В матричной форме: пусть $\mathbf{P} \in \mathbb{R}^{n_m \times (H_p W_p)}$ - прототипы, развёрнутые по пространственным индексам; тогда вектор логитов маски для детекции i есть $\mathbf{a}_i^\top \mathbf{P}$, после σ и обратного преобразования в форму $H_p \times W_p$ получают $\tilde{M}_i$. Далее маску обрезают по ограничивающему прямоугольнику детекции b_i , при необходимости бинаризуют по порогу (например, 0.5) и масштабируют до размера кадра или бокса для получения финальной пиксельной маски M_i .

На выходе сегментатора для каждого объекта имеем тройку (b_i, c_i, M_i) : бокс, класс и маска контура. Обучение сегментации ведётся совместно с детекцией; в функцию потерь входят слагаемые для боксов, классификации и для масок (например, бинарная кросс-энтропия или dice между предсказанной и целевой маской в области объекта).

Ниже на рисунке 3 представлена архитектура YOLO-seg:

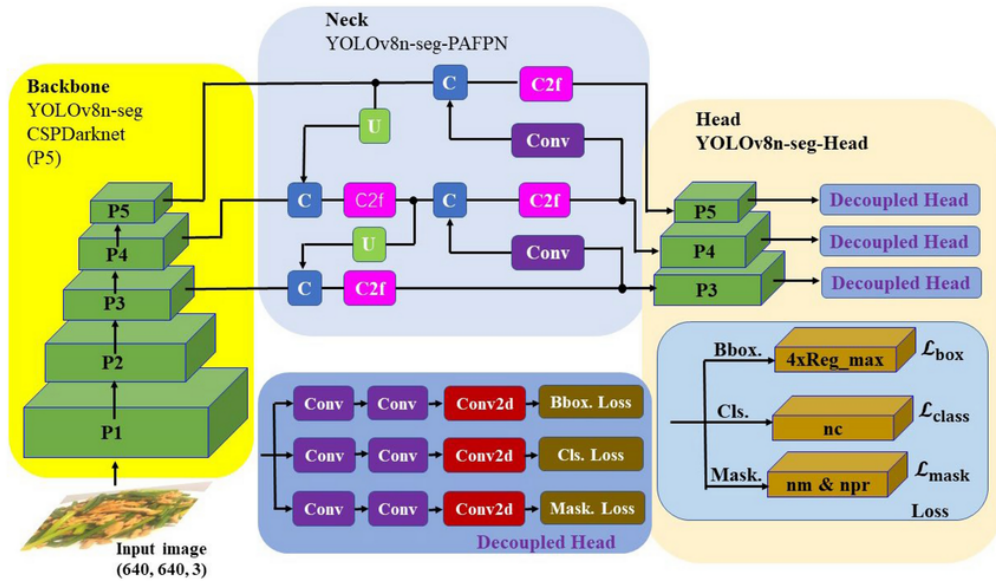


Рис. 3: Архитектура YOLO-seg на примере YOLOv8n-seg

7 Подготовка датасета

В качестве выборки для обучения выбран открытый датасет RailSem19: семантическая сегментация сцен железнодорожной инфраструктуры. Датасет содержит 8500 изображений с пиксельной разметкой. В состав входят: сами изображения; разметка в формате JSON; маски в формате `uint8`; конфигурационный JSON с объявлением классов.

Маска семантической сегментации хранится как одноканальное изображение, в котором каждому пикселю соответствует один байт (тип `uint8`) - индекс класса. Значение пикселя $p(h, w) \in \{0, 1, \dots, N_{\text{cls}} - 1\}$ указывает принадлежность пикселя (h, w) одному из N_{cls} классов; при $N_{\text{cls}} \leq 256$ одного байта достаточно. Такой формат компактен по памяти (один байт на пиксель вместо трёх каналов в RGB-маске), удобен для многоклассовой сегментации и напрямую подходит для обучения моделей типа SegFormer или DeepLabV3+: функция потерь (кросс-энтропия) ожидает на входах целочисленные метки классов по пикселям. В RailSem19 маски в `uint8` позволяют однозначно задать семантическую разметку для всех 19 классов.

В датасете используется 19 классов, объединённых в три группы: рельсы (5 классов), городская среда (11 классов), транспорт (4 класса). Конфигурационный JSON задаёт соответствие между индексами и именами классов и используется при загрузке разметки и при конвертации в другие форматы.

Обучение модели YOLO-seg на таких данных в исходном виде невозможно из-за различия форматов. Каждый объект задаётся отдельно, разметка хранится в текстовых файлах (один `.txt` на изображение), в каждой строке - один объект в виде

`<класс> <x_1> <y_1> <x_2> <y_2> ...<x_n> <y_n>`,

где координаты (x_i, y_i) - вершины полигона контура объекта в нормализованных координатах (относительно ширины и высоты изображения, значения в $[0, 1]$). Минимум три точки на объект; число вершин у разных объектов может отличаться. Таким образом, YOLO-seg ожидает не пиксельную карту классов (`uint8` маску), а список полигонов по объектам. При обучении контур каждого объекта по этим вершинам растеризуется в бинарную маску в области ограничивающего прямоугольника; по этой маске считается слагаемое потерь для масок (например, бинарная кросс-энтропия или dice). Итоговая маска на инференсе строится из прототипов и коэффициентов; формат разметки при обучении - именно полигоны, чтобы компактно хранить контуры и однозначно сопоставлять каждую маску конкретному объекту (детекции).

Для приведения датасета RailSem19 к формату, пригодному для обучения YOLO-seg, используется репозиторий JSON2YOLO от Ultralytics. Он автоматически преобразует исходные аннотации (JSON и маски) в структуру каталогов с подпапками `images/` и `labels/`: для каждого изображения создаётся текстовый файл с тем же именем, в котором для каждого объекта указаны индекс класса и нормализованные координаты вершин полигона в одну строку. Дополнительно задаётся YAML-конфигурация датасета с путями к `train/` и `val/` и словарём имён классов. После конвертации датасет можно использовать для обучения YOLO-seg и оценки качества сегментации в формате попиксельной сегментации.

Ниже представлен пример кода конвертации датасета на основе JSON, в формат обучения YOLO:

За основу берется метод `convert_coco` из библиотеки `ultralytics`.

```
1 from ultralytics.data.converter import convert_coco
2
3 convert_coco(
4     labels_dir='PATH/to/annotations/',
5     use_segments=True
6 )
```

8 Стандартное обучение сегментаторов

Под стандартным обучением в работе, понимается стандартное деление датасета на обучающую, валидационную и тестовую части выборки. В последствие для выбранной модели сегментатора, будет использоваться кросс-валидация с методом *leave one out*.

Подбор гиперпараметров средствами Optune - долгая и тяжеловесная задача для видеопамати. Было принято решение использовать более легкий метод подбора гиперпараметров методом tune, который предоставляет ultralytics для моделей YOLO-seg.

8.1 Генетический алгоритм tune

Метод tune от ultralytics представляет собой эволюционный алгоритм, направленный на максимизацию метрик качества модели *mAP*, *accuracy* и других метрик в зависимости от модели. В основе процесса лежит использование библиотеки Ray Tune, для перебора пространства гиперпараметров.

Формально настройка гиперпараметров - задача оптимизации:

$$\theta^* = \arg \max_{\theta \in \Theta} F(M(\theta), D_{val}),$$

где θ - вектор гиперпараметров, Θ - пространство поиска (диапазон допустимых значений для гиперпараметров), $M(\theta)$ - модель сегментатора обучающаяся на векторе θ , D_{val} - валидационный набор данных, F - целевая функция (основная метрика максимизации), θ^* - оптимальный набор гиперпараметров.

Во время каждой эволюции (trial), внутри настройки модель оптимизирует веса ω через функцию потерь $\mathcal{L}$

Модель YOLO, имеет единый прогон по нейросети которая: выделяет, классифицирует, заносит в рамки сегмент, и выглядит как сумма всех функций потерь:

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{seg} + \lambda_2 \mathcal{L}_{cls} + \lambda_3 \mathcal{L}_{box}$$

где коэффициенты λ_i - так же могут настраиваться.

Чтобы алгоритм выявлял лучшее решение, задается взвешанная сумма метрик, которая является скалярным произведением вектора результатов на результаты весов:

$$F(h) = \sum_{i=1}^n w_i \cdot m_i$$

В YOLO-seg, фитнес функция задается как $F = 0.1 \cdot mAP@50 + 0.9 \cdot mAP@50 - 95$.

Инициализация происходит следующим образом:

N - набор гиперпараметров, на каждой итерации алгоритм выбирает k - лучших родителей у котолрых значение $F(h) = \arg \max F(h)$, остальные особи отбрасываются.

Новое поколение создается оператором мутации путем добавления случайного шума к параметрам лучших родителей:

$$\forall h_j : h_j^{new} = h_j^{parent} \cdot M$$

M - множитель мутации определяется через распределение:

$$M = clip(1 + \mathcal{N}(0, \sigma^2), min, max),$$

где σ - коэффициент интенсивности мутации с затуханием, $clip$ - функция ограничения чтобы параметр не вышел за границы (например что бы псевдо-вероятность не превышала 1).

Кроссовер как скрещивание двух родителей выглядит как линейная комбинация двух родителей P_1, P_2 :

$$h^{child} = \alpha P_1 + (1 - \alpha) P_2$$

При этом, α случайное число. В итоге ребенок наследует каждый ген (параметр) от одного из родителей с вероятности $\frac{1}{2}$.

Итерации при этом являются цепью Маркова, где следующее состояние популяции H_{t+1} зависит от текущего H_t

Вычисляется $F(h_i) \forall i \in 1, \dots, N$, далее идет сортировка такая что $F(h_1) \geq F(h_2) \geq \dots \geq F(h_N)$, тогда следующий $h_i^{t+1} = Mutate(Select(H_t))$. Остановка происходит либо вручную пока $t = T$, либо по заданной точности ε .

Для реализации подбора гиперпараметров, необходимо воспользоваться библиотекой ultralytics:

```
1 from ultralytics import YOLO
2
3 model = YOLO("yolo11s-seg.pt")
4
5 results = model.tune(
6     data="PATH",
7     epochs=30,
8     iterations=100,
9     optimizer="AdamW",    # OPTIMIZER
10    plots=False,
11    save=False,
12    val=True
13 )
```

8.2 Результаты подбора гиперпараметров

По результатам подбора, использовалось 30 эпох на 20 поколений, что заняло 35 часов. На рисунке 4, представлены диаграммы рассеивания гиперпараметров. Черный крестик, обозначает лучший гиперпараметр из всех использованных.

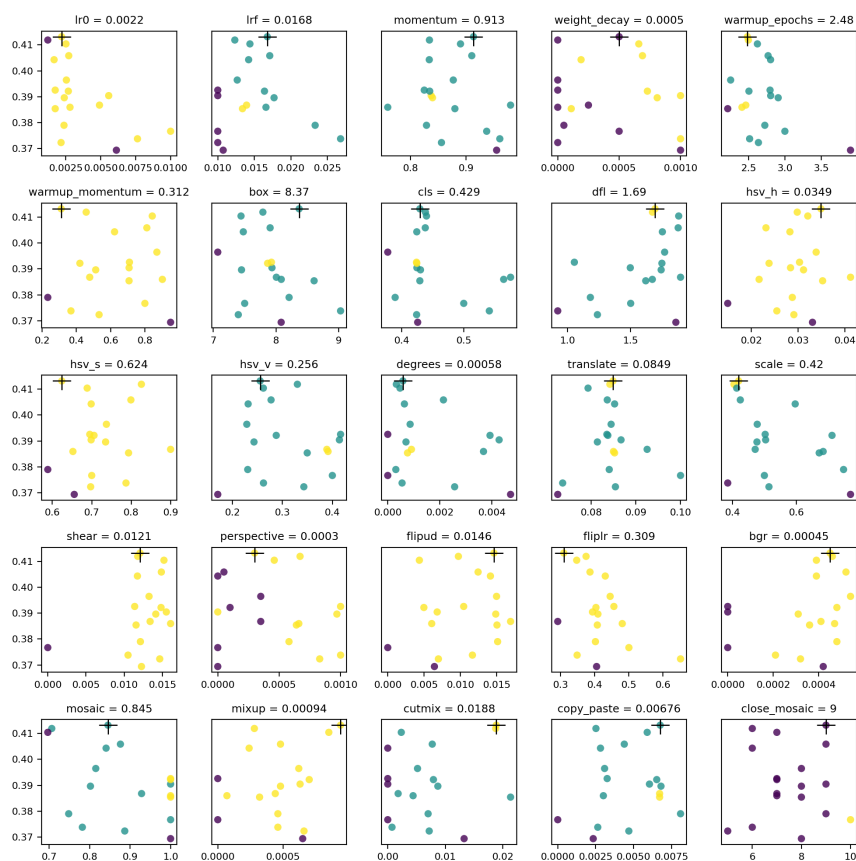


Рис. 4: Результаты подбора гиперпараметров

Ниже в таблице 1 представлены полученные гиперпараметры для обучения

Таблица 1: Таблица 1. Полученные гиперпараметры (часть 1)

№	fit	lr0	lrf	mom	wd	warmEp	warmMom	box	cls	df	hsv_h	hsv_s	hsv_v	deg
1	0.377	0.010	0.010	0.937	0.0005	3.0	0.80	7.50	0.50	1.50	0.015	0.70	0.40	0.0
2	0.390	0.0056	0.010	0.837	0.001	2.80	0.71	7.94	0.42	1.50	0.028	0.70	0.41	0.004
3	0.393	0.0017	0.010	0.825	0.0	2.79	0.71	7.93	0.42	1.05	0.030	0.70	0.42	0.0
4	0.404	0.0017	0.014	0.834	0.0002	2.80	0.62	7.48	0.42	1.75	0.028	0.70	0.23	0.001
5	0.412	0.0012	0.012	0.834	0.0	2.49	0.46	7.79	0.44	1.67	0.030	0.83	0.33	0.0003
6	0.372	0.0022	0.010	0.856	0.0	2.63	0.53	7.40	0.42	1.24	0.029	0.70	0.34	0.003
7	0.392	0.0027	0.016	0.834	0.0007	2.50	0.42	7.87	0.42	1.75	0.024	0.71	0.29	0.004
8	0.413	0.0022	0.017	0.913	0.0005	2.48	0.31	8.37	0.43	1.69	0.035	0.62	0.26	0.001
9	0.406	0.0027	0.017	0.910	0.0007	2.77	0.81	7.91	0.44	1.88	0.023	0.80	0.28	0.002
10	0.374	0.0076	0.027	0.961	0.001	2.51	0.37	9.03	0.54	0.92	0.025	0.79	0.26	0.001
11	0.387	0.0049	0.014	0.98	0.0003	2.46	0.48	8.01	0.58	1.90	0.041	0.90	0.39	0.001
12	0.390	0.0024	0.018	0.840	0.0008	2.91	0.51	7.45	0.43	1.74	0.031	0.74	0.24	0.001
13	0.386	0.0028	0.017	0.759	0.0	2.41	0.90	8.08	0.56	1.67	0.022	0.65	0.39	0.004
14	0.379	0.0024	0.023	0.829	0.0001	2.72	0.23	8.21	0.39	1.18	0.029	0.59	0.23	0.0003
15	0.369	0.0061	0.011	0.955	0.001	3.91	0.95	8.08	0.43	1.86	0.033	0.66	0.17	0.005
16	0.410	0.0025	0.014	0.890	0.0007	2.62	0.84	7.44	0.44	1.88	0.032	0.69	0.26	0.0005
17	0.0	0.0021	0.012	0.925	0.0	2.56	0.41	4.72	0.44	2.39	0.034	0.80	0.22	0.0003
18	0.396	0.0025	0.013	0.877	0.0	2.25	0.87	7.07	0.38	1.77	0.034	0.74	0.23	0.001
19	0.385	0.0017	0.013	0.880	0.0001	2.21	0.71	8.61	0.43	1.62	0.035	0.79	0.35	0.001
20	0.0	0.0017	0.014	0.851	0.0001	2.96	0.89	7.09	0.45	2.06	0.033	0.64	0.32	0.002

№	transl	scale	shear	persp	flipud	fliplr	bgr	mosaic	mixup	cutmix	copy_p	close
1	0.10	0.50	0.0	0.0	0.0	0.50	0.0	1.0	0.0	0.0	0.0	10
2	0.087	0.50	0.016	0.0	0.007	0.39	0.0	1.0	0.0006	0.0	0.006	9
3	0.084	0.50	0.011	0.001	0.010	0.46	0.0	1.0	0.0	0.0	0.003	7
4	0.085	0.60	0.012	0.0	0.014	0.43	0.0004	0.84	0.0002	0.0	0.003	6
5	0.084	0.41	0.012	0.001	0.010	0.37	0.0005	0.71	0.0003	0.019	0.003	6
6	0.085	0.51	0.015	0.001	0.007	0.65	0.0003	0.89	0.0007	0.007	0.005	5
7	0.084	0.71	0.015	0.0001	0.005	0.40	0.0005	1.0	0.0007	0.008	0.007	7
8	0.085	0.42	0.012	0.0003	0.015	0.31	0.0005	0.85	0.0009	0.019	0.007	9
9	0.084	0.42	0.015	0.0001	0.012	0.39	0.0005	0.88	0.0005	0.008	0.004	9
10	0.074	0.39	0.011	0.001	0.012	0.35	0.0002	0.78	0.0005	0.001	0.003	6
11	0.093	0.47	0.013	0.0004	0.017	0.29	0.0004	0.93	0.0004	0.002	0.007	7
12	0.081	0.48	0.014	0.001	0.015	0.41	0.0003	0.80	0.0005	0.009	0.007	8
13	0.085	0.68	0.016	0.001	0.006	0.48	0.0005	1.0	0.0001	0.004	0.003	7
14	0.084	0.75	0.012	0.001	0.015	0.40	0.0005	0.75	0.0005	0.007	0.008	9
15	0.073	0.77	0.012	0.0	0.006	0.41	0.0004	1.0	0.0007	0.013	0.002	8
16	0.079	0.41	0.015	0.0005	0.004	0.35	0.0004	0.70	0.0009	0.002	0.006	7
17	0.090	0.45	0.012	0.001	0.010	0.36	0.0004	0.50	0.0008	0.010	0.007	6
18	0.084	0.48	0.013	0.0004	0.015	0.45	0.0005	0.82	0.0006	0.005	0.003	8
19	0.085	0.67	0.012	0.001	0.015	0.41	0.0004	1.0	0.0003	0.021	0.007	8
20	0.086	0.61	0.018	0.0	0.004	0.43	0.0007	0.66	0.0003	0.013	0.006	8

Обозначения: fit - fitness; lr0, lrf - параметры learning rate; mom - momentum; wd - weight decay; warmUp, warmMom - warmup; box, cls, dfl - веса потерь; hsv_h/s/v - аугментация HSV; deg - degrees; transl - translate; persp - perspective; copy_p - copy_paste; close - close_mosaic.

Окончательные результаты гиперпараметров представлены в таблице 2:

Таблица 2: Лучший результат из подбора гиперпараметров

Параметр	Значение	Параметр	Значение	Параметр	Значение
lr0	0.0022	lrf	0.01678	momentum	0.91324
weight_decay	0.0005	warmup_epochs	2.48151	warmup_momentum	0.31178
box	8.369	cls	0.42917	dfl	1.69298
hsv_h	0.03487	hsv_s	0.62407	hsv_v	0.25582
degrees	0.00058	translate	0.0849	scale	0.41977
shear	0.01207	perspective	0.0003	flipud	0.01461
fliplr	0.30946	bgr	0.00045	mosaic	0.8451
mixup	0.00094	cutmix	0.01884	copy_paste	0.00676
close_mosaic	9				

8.3 Выбор метода оптимизации

Подбор гиперпараметров, касаются архитектуры моделей ultralytics, эти параметры будут использованы для обучения и выбора модели между версиями и конфигурациями YOLO-seg:

- YOLOv8n-seg;
- YOLOv8s-seg;
- YOLOv11s-seg;
- YOLO26s-seg.

Модель YOLOv8n-seg, является самой легковесной моделью, которая ориентирована на мобильные устройства. YOLOv8s-seg, тяжелее относительно YOLOv8n-seg, но она так же ориентирована на мобильные устройства с более мощным аппаратным обеспечением. Модель YOLOv11s-seg, является более продвинутой моделью относительно YOLOv8s-seg, использующая более мощный и оптимизированный энкодер. YOLO26s-seg, современная модель, вышедшая в 2026 году, однако из источников, сообщается о том, что модель не ориентирована на специфические датасеты, и более ориентирована на датасет типа COCO, которую используют ultralytics, для оценки точности модели.

Для реализации обучения, изначально к гиперпараметрам, необходимо подобрать метод оптимизации минимизации функции потерь, так как этот метод будет использоваться для обучения всех моделей.

8.4 Эволюционный алгоритм CMA-ES

В рассматриваемых вариантах для минимизации из генетических алгоритмов, был выбран алгоритм CMA-ES.

CMA-ES (Covariance Matrix Adaptation - Evolution Strategy) - это итеративный стохастический алгоритм оптимизации второго порядка, для поиска минимума в сложных пространствах.

Входными данными алгоритма являются:

- Целевая функция которую необходимо минимизировать - это функция потерь;
- Начальная точка: $m \in \mathbb{R}^n$ - центр распределения (начальное приближение);
- Начальный шаг: $\sigma \in \mathbb{R}_+$ - размер области поиска;
- Размер популяции: λ - количество точек генерируемых в каждой итерации;

Основной цикл, состоит из пяти частей - в начале генерируется λ векторов из многомерного нормального распределения $x_k \sim \mathcal{N}(0, C)$, где C - ковариационная матрица.

Далее вычисляются значения целевой функции $f(x_k)$, для всех точек. Точки сортируются по значению, и отбираются $\mu \approx \frac{\lambda}{2}$ лучших кандидатов.

После вычисляется среднее значение m , как взвешенная сумма лучших точек, что позволяет смещать центр поиска в окрестности минимума.

Если последовательные шаги среднего значения сонаправлены, то σ увеличивается и алгоритм ускоряется, если шаги «гасят» друг друга, то σ уменьшается, что означает, что алгоритм сходится.

Матрица C , обновляется так, что бы увеличить вероятность генерации точек в направлениях где ранее было достигнуто лучшее значение. Таким образом пространство поиска деформируется в вытянутый эллипсоид, оси которого соответствуют собственным векторам C . Процесс является аппроксимацией обратной матрицы Гесса.

На выходе, алгоритм возвращает вектор x_k , который является точкой минимума функции потерь, а матрица C описывает локальную структуру функции.

На рисунке 5, изображена блок-схема алгоритма:

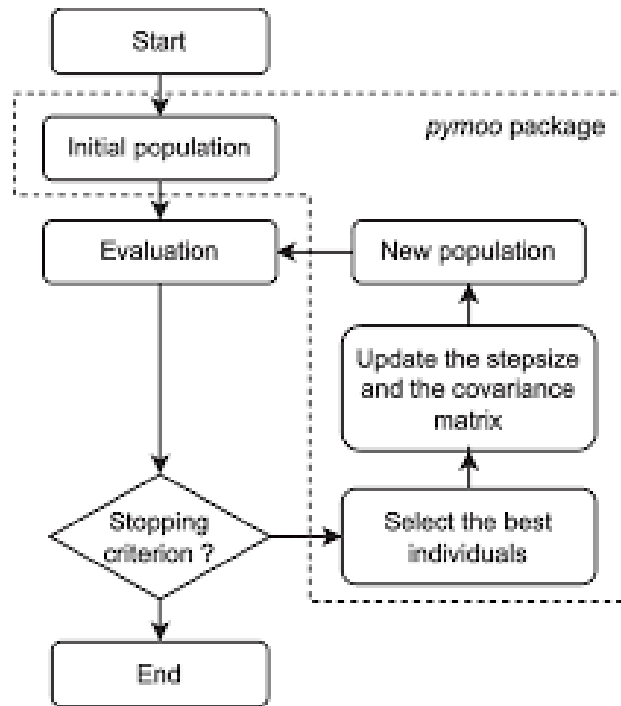


Рис. 5: Блок-схема CMA-ES

Сложность алгоритма составляет по памяти $O(n^2)$, где n - количество параметров.

Модель YOLOv8n-seg, хоть и является самой маленькой моделью, но имеет 3.4 миллионов параметров. Что означает, что при классической реализации алгоритма CMA-ES, в VRAM, придется хранить $\approx 3400000^2 \approx 11560000000000$ параметров. В теории, данный алгоритм при инициализации популяции, должен быть прерван на стадии инициализации. На практике можно применить оптимизированный алгоритм CMA-ES, из библиотеки torch - eye.

Для начала нужно сформировать вектор из параметров модели YOLOv8s-seg:

```

1  import torch
2  import numpy as np
3  from ultralytics import YOLO
4
5  model = YOLO("yolov8n-seg.pt")
6
7  trainable_params = [p for p in model.model.parameters() if p.
8                      requires_grad]
9  N = sum(p.numel() for p in trainable_params)
10
11 try:
12     m = torch.cat([p.view(-1) for p in trainable_params]).detach().numpy()
13     C = np.eye(N, dtype=np.float32)
14 except MemoryError:
15     print("Oops! Out of Memory")
  
```

Результатом вывода оказалось строка: «Oops! Out of Memory», что подтверждает, что памяти не хватит под модели обучения.

Для реализации минимизации функции потерь на практике используются методы градиентного спуска: стохастический градиентный спуск либо адаптивная оценка моментов с разделенным распадом весов (AdamW).

8.5 SGD

Метод SGD - стохастический градиентный спуск, основан на обновлении весов по направлению антиградиента:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla f(\theta_t)$$

При этом в YOLO используется добавление инерции, для ускорения спуска (этот метод является инерционным). При этом алгоритм становится таким:

- $v_{t+1} = \mu v_t + \nabla f(\theta_t)$
- $\theta_{t+1} = \theta_t - \eta v_{t+1}$

Где μ - коэффициент инерции, η - скорость обучения (learning rate в ultralytics).

8.6 AdamW

Данный метод, вычисляет индивидуальный шаг обучения для каждого параметра, используя оценки первого и второго момента градиента.

Алгоритм выглядит следующим образом:

- Обновление моментов:
 1. $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f(\theta_t)$ - среднее значение градиента
 2. $v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla f(\theta_t))^2$ - дисперсия
- Корректировка смещения: $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$ и $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
- Обновление весов: $\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$

Теоретически, метод AdamW является более точным, чем SGD, хотя и более ресурсоемкий.

Принято решение использовать метод AdamW, для обеспечения более точной минимизации функции потерь.

9 Обучение сегментаторов

Для реализации обучения сегментаторов, используется библиотека `ultralytics`, для этого используется метод `train()` из библиотеки, которая принимает на вход полученные гиперпараметры из главы 8.2.

Для начала нужно импортировать необходимые библиотеки, и объявить константы для разбиения датасета на данные для обучения, валидации и тестирования.

```
1 import os
2 import random
3 import yaml
4 from pathlib import Path
5 from ultralytics import YOLO, settings
6
7 os.environ["TORCH_FORCE_WEIGHTS_ONLY_LOAD"] = "0"
8
9 _SCRIPT_DIR = Path(__file__).resolve().parent
10 DIR_YOLO_DATASET = _SCRIPT_DIR / "yolo_dataset"
11 DATA_YAML = DIR_YOLO_DATASET / "data.yaml"
12 YOLO_DATA_PATH = DIR_YOLO_DATASET
13 IMG_SRC = None
14 LBL_SRC = None
15
16 TRAIN_SIZE = 7000
17 VAL_SIZE = 1000
18 TEST_SIZE = 500
```

После чего необходимо создать словарь для объявления классов и их индексов:

```
1 CLASS_NAMES = {
2     0: "road", 1: "sidewalk", 2: "construction", 3: "tram-track", 4: "
3         fence",
4     5: "pole", 6: "traffic-light", 7: "traffic-sign", 8: "vegetation",
5     9: "terrain",
6     10: "sky", 11: "human", 12: "rail-track", 13: "car", 14: "truck",
7     15: "trackbed", 16: "on-rails", 17: "rail-raised", 18: "rail-
8         embedded",
9 }
10 }
```

Далее реализуется метод для разбиения датасета на данные для обучения, валидации и тестирования:

```
1 def _resolve_data_paths():
2     global IMG_SRC, LBL_SRC
3     if IMG_SRC is not None and LBL_SRC is not None:
4         return
5     exts = (".jpg", ".jpeg", ".png", ".bmp")
6     if DATA_YAML.exists():
7         with open(DATA_YAML, encoding="utf-8") as f:
8             cfg = yaml.safe_load(f)
9             base = Path(cfg.get("path", "."))
10            if not base.is_absolute():
11                base = (DATA_YAML.parent / base).resolve()
12            train_rel = cfg.get("train", "images/rs19_val")
13            img_dir = base / train_rel
14            lbl_dir = base / "labels" / Path(train_rel).name
15            if img_dir.is_dir() and lbl_dir.is_dir():
16                n = [f.stem for f in lbl_dir.glob("*.txt") if any((img_dir / f"{
17                    f.stem}{e}").exists() for e in exts))]
18            if n:
```

```

18         IMG_SRC = img_dir
19         LBL_SRC = lbl_dir
20         return
21     for part in ("rs19_val", "train"):
22         img_dir = DIR_YOLO_DATASET / "images" / part
23         lbl_dir = DIR_YOLO_DATASET / "labels" / part
24         if img_dir.is_dir() and lbl_dir.is_dir():
25             n = [f.stem for f in lbl_dir.glob("*.txt") if any((img_dir / f"{
26                 f.stem}{e}").exists() for e in exts))]
27             if n:
28                 IMG_SRC = img_dir
29                 LBL_SRC = lbl_dir
30                 return
31     raise FileNotFoundError(
32         f"Images and labels not found. Create {DIR_YOLO_DATASET}/data.yaml
33         with path and train "
34         f"or put data in {DIR_YOLO_DATASET}/images/rs19_val and {
35         DIR_YOLO_DATASET}/labels/rs19_val (or images/train, labels/train
36         )."
37     )

```

Главный метод для обучения использует в теле метод train:

```

1     model.train(
2         data=str(YOLO_DATA_PATH / "data.yaml"),
3         epochs=150,
4         imgsiz=800,
5         batch=8,
6         amp=True,
7         optimizer="AdamW",
8         lr0=0.0022,
9         lrf=0.01678,
10        momentum=0.91324,
11        weight_decay=0.0005,
12        warmup_epochs=2.48151,
13        warmup_momentum=0.31178,
14        box=8.369,
15        cls=0.42917,
16        dfl=1.69298,
17        hsv_h=0.03487,
18        hsv_s=0.62407,
19        hsv_v=0.25582,
20        degrees=0.00058,
21        translate=0.0849,
22        scale=0.41977,
23        shear=0.01207,
24        perspective=0.0003,
25        flipud=0.01461,
26        fliplr=0.30946,
27        bgr=0.00045,
28        mosaic=0.8451,
29        mixup=0.00094,
30        close_mosaic=9,
31        mask_ratio=4,
32        overlap_mask=True,
33        label_smoothing=0.1
34    )

```

Гиперпараметр mask_ratio, был изменен с 1 на 4, так как операция сглаживания масок приводила к ошибке CUDA out of memory.

Объявлено 150 эпох обучения, для получения первичных результатов.

Данный метод обучения, подходит для всех моделей сегментаторов YOLO-seg, так как было сказано ранее, что гиперпараметры являются общими из-за схожих архитектур моделей.

10 Результаты обучения

Ниже на рисунках 6 - 13 представлены графики результатов обучения сегментаторов YOLO-seg. Каждый график представляет собой значения функции потерь на эпохах обучения, где представлены слева направо: потери присваивания рамок, потери сегментации, потери классификации, потери краев рамок, потери семантической сегментации (не используется на валидации)

Далее идут графики метрик - точность масок (precision M), точность рамок (precision B), полнота масок (recall M), полнота рамок (recall B), средняя точность детекции при условиях что рамка совпала с истинной хотя бы на 50 % (B, M - для рамок и масок) mAP@50-95 - среднее при разных порогах совпадения рамок (B) и масок (M) от 50 % до 95 %.

В датасете RailSem19, на каждом изображении есть рельсы, но не в каждом изображении есть остальные классы! Это объясняет причину плохого распознавания остальных классов.

10.1 Результаты обучения сегментатора YOLOv8n-seg

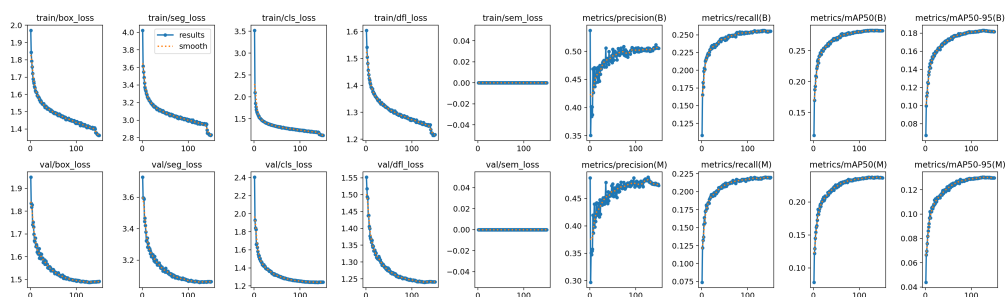


Рис. 6: Результаты обучения сегментатора YOLOv8n-seg

Как видно из рисунка 6, модель YOLOv8n-seg, при обучении вышло на плато. Так же наблюдается резкие спады потерь на всех функциях после 130 эпохи обучения. При этом на валидации после ≈ 110 эпохи до 140 эпохи сработало останова так как точность минимизации не изменялась значительно 30 эпох подряд.

Таким образом модель YOLOv8n-seg, обучилась полностью. Однако если взглянуть на матрицу ошибок, видно что модель плохо распознает трамвайные пути и рельсы. Всего 55% для трамвайных путей, 54% для путей и 47% для железной дороги как видно из рисунка 7.

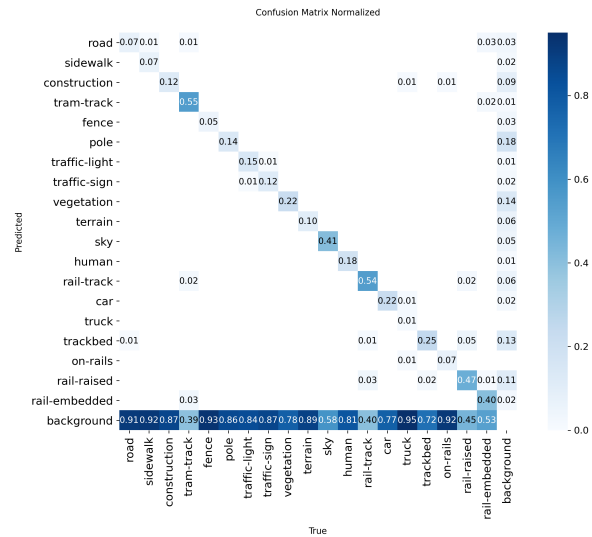


Рис. 7: Матрица ошибок обучения сегментатора YOLOv8n-seg

10.2 Результаты обучения сегментатора YOLOv8s-seg

Модель YOLOv8s-seg, обучалась все 150 эпох. При этом, на всех графиках обучения наблюдаются признаки переобучения, а на валидации плавное повышение значений функций потерь.

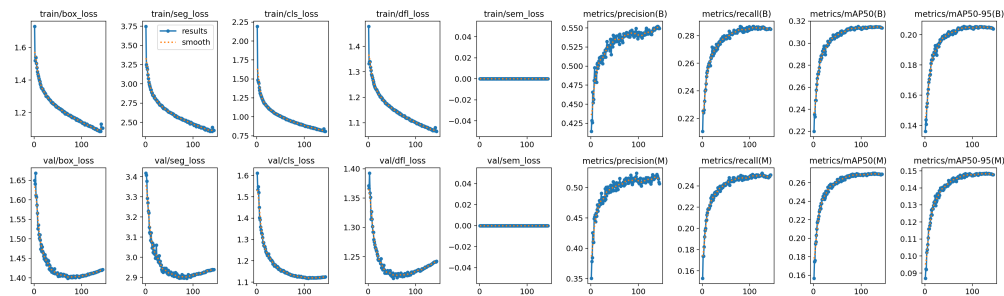


Рис. 8: Результаты обучения сегментатора YOLOv8s-seg

По матрице ошибок, видно что модель YOLOv8s-seg, чуть лучше распознает трамвайные пути и рельсы. 59% для трамвайных путей, 60% для путей в целом и 51% для железной дороги как видно из рисунка 9.

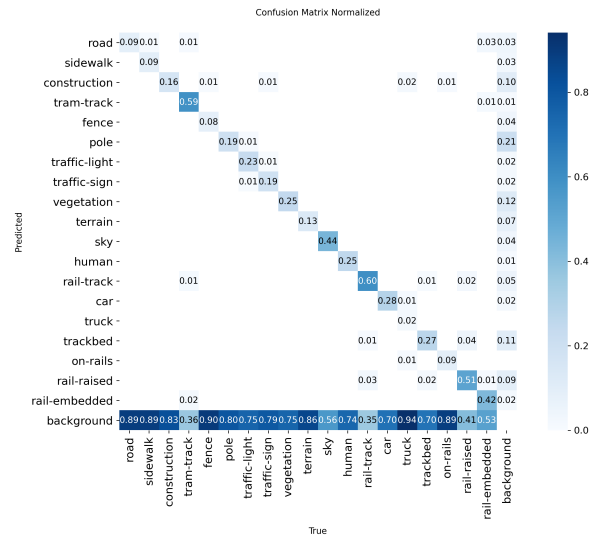


Рис. 9: Матрица ошибок обучения сегментатора YOLOv8s-seg

10.3 Результаты обучения сегментатора YOLOv11s-seg

Хорошие результаты показала модель YOLOv11s-seg, которая так же обучалась все 150 эпох. Модель является более лучшей версией YOLOv8s-seg, скорее всего показатели выше так как точность сегментации выше из-за того что рельсы на изображениях в дальнем виде отображаются как несколько пикселей которые YOLOv8s-seg и YOLOv8n-seg игнорировали.

По графикам обучения, видно что модель еще могла обучаться, так как и на валидации и на обучении наблюдаются признаки недообучения. Все графики (кроме графиков метрик точностей) стремятся вниз и не выходят на плато.

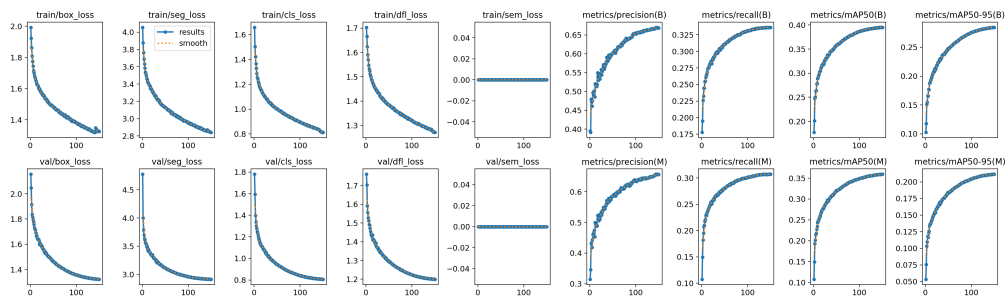


Рис. 10: Результаты обучения сегментатора YOLOv11s-seg

По матрице ошибок, YOLOv11s-seg, показала удовлетворительные результаты которые подходят для критерия точности: 71% для трамвайных путей, 68% для путей в целом и 65% для железной дороги (см. рисунок 11).

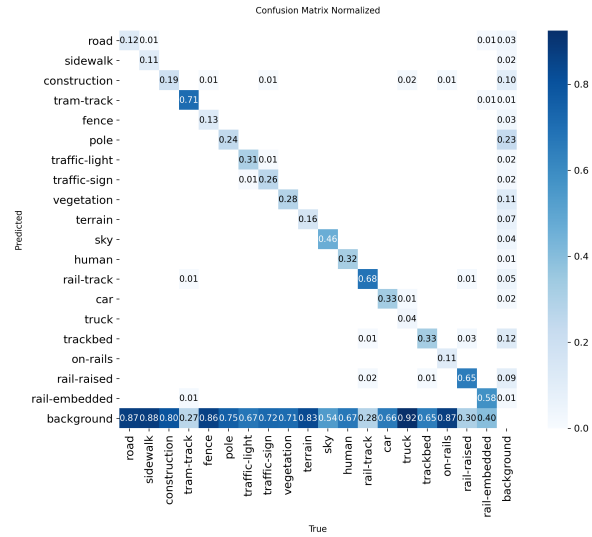


Рис. 11: Матрица ошибок обучения сегментатора YOLOv11s-seg

10.4 Результаты обучения сегментатора YOLO26s-seg

Модель YOLO26s-seg, обучалась так же все 150 эпох. На графиков так же наблюдается признаки недообучения на валидации. При этом на обучении наблюдаются резкие скачки на обучении примерно все на 140 эпохи. Возможно модели нужно было обучаться дольше, но так же возможно что новая модель действительно не подходит для датасета RailSem19.

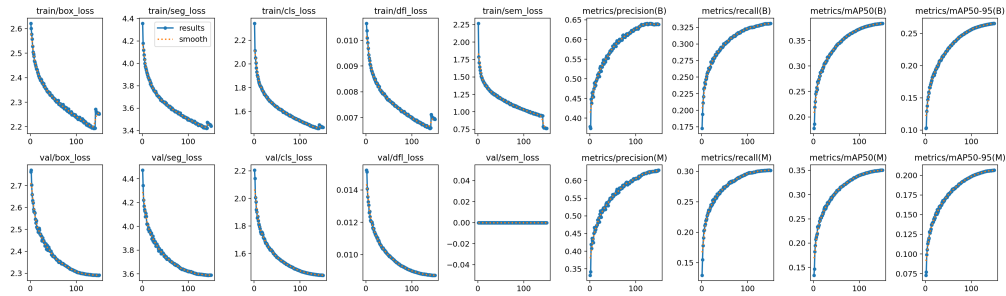


Рис. 12: Результаты обучения сегментатора YOLO26s-seg

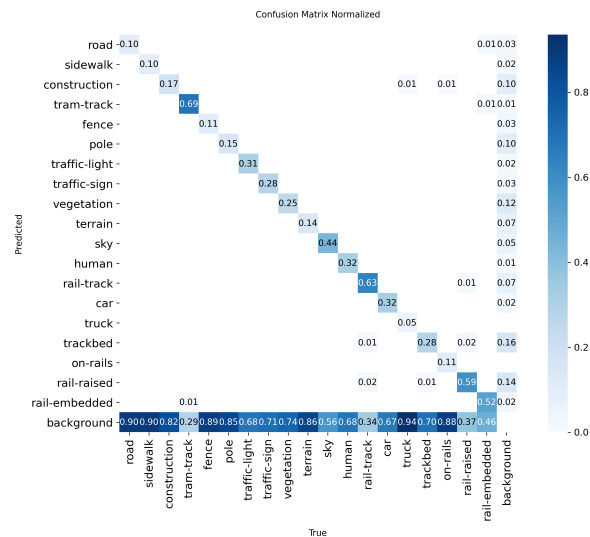


Рис. 13: Матрица ошибок обучения сегментатора YOLO26s-seg

По матрице ошибок результаты хуже чем у YOLOv11s-seg, но лучше чем у YOLOv8s-seg и YOLOv8n-seg: 69% для трамвайных путей, 63% для путей в целом и 59% для железной дороги (см. рисунок 13).

10.5 Результаты обучения SegFormer B0

Для обучения сегментатора SegFormer, были найдены оптимальные гиперпараметры находящиеся в открытом доступе из источника [10].

После обучения сегментатора, были получены следующие результаты:

На рисунке 14 представлены значения функции потерь на эпохах обучения SegFormer B0, сегментатор обучался 122 эпохи, после чего сработал останов обучения так как значения функции не менялись 15 эпох.

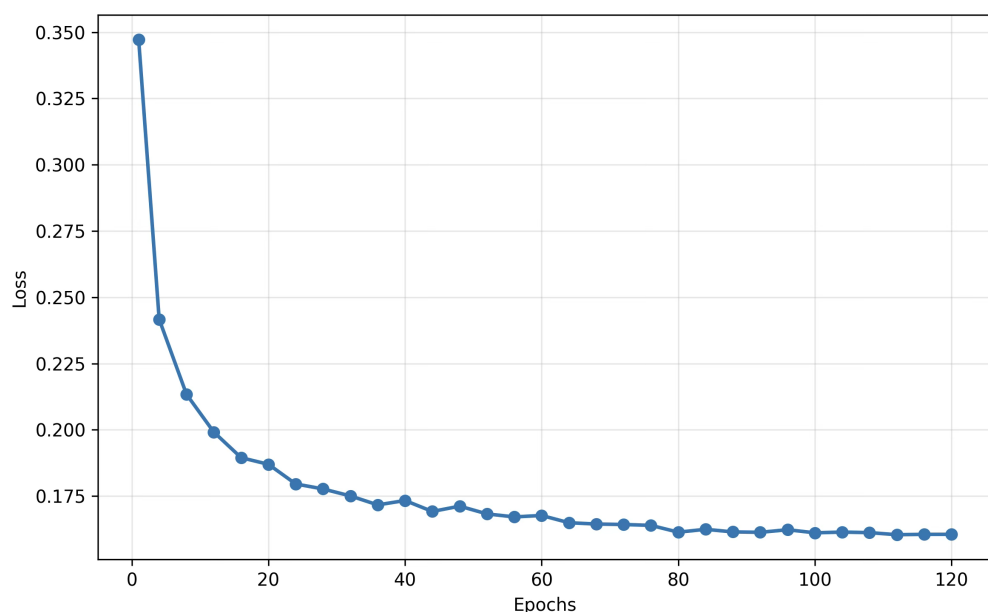


Рис. 14: Значения функции потерь на эпохах обучения SegFormer B0

На рисунке 15 представлены значения learning rate на эпохах обучения SegFormer B0:

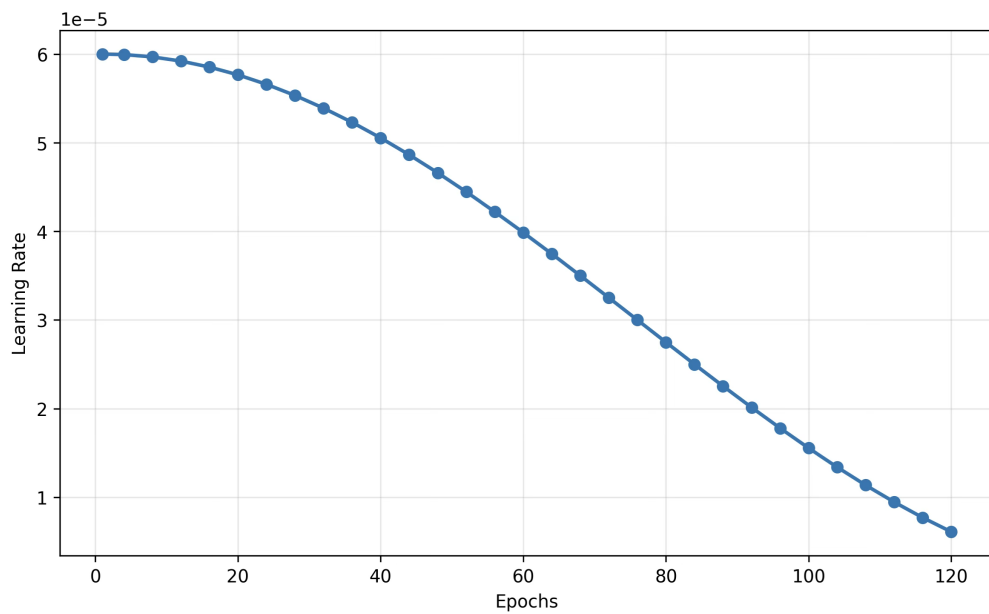


Рис. 15: Значения learning rate на эпохах обучения SegFormer B0

Как видно, скорость обучения постепенно снижалась до последней эпохи обучения при этом могло бы спускаться дальше но остановилась на эпохе 122.

10.6 DeepLabV3+ ResNet101

Обучение сегментатора DeepLabV3+ с энкодером ResNet101, было проведено с помощью библиотеки PyTorch.

Веса модели находились в открытом доступе [13], что не потребовало дополнительного обучения модели.

11 Анализ результатов

Исходя из результатов быстрого подбора гиперпараметров и обучения сегментаторов был сделан следующий вывод:

- **DeepLabV3+ с энкодером ResNet101** - оказался слишком медленным и не точным для сегментации трамвайных путей;
- **SegFormer B0** - оказался не точным для сегментации и слишком тяжелым для обучения, возможно модели B3, B5, были более точными но, эти модели не приспособлены на работу в реальном времени;
- **YOLOv8n-seg** - оказался самой быстрой моделью однако показал плохие результаты на валидации и не смог пройти критерий точности;
- **YOLOv8s-seg** - точность оказалась лучше чем у YOLOv8n-seg, но так же не смогла пройти критерий точности, при этом были явные признаки переобучения;
- **YOLOv11s-seg** - показала лучшие результаты как по скорости так и по точности распознавания, модель явно стоит дообучать;
- **YOLOv26s-seg** - показала результаты хуже чем у YOLOv11s-seg, но лучше чем у других моделей, однако модель либо не подходит для выбранного датасета, либо для новой модели нужно искать новые гиперпараметры. Однако как видно из результатов, модели YOLOv11s-seg вполне хватает для решения поставленной задачи.

Ниже на рисунках 16 - 21, представлены результаты сегментации а так же скорость их инференса:

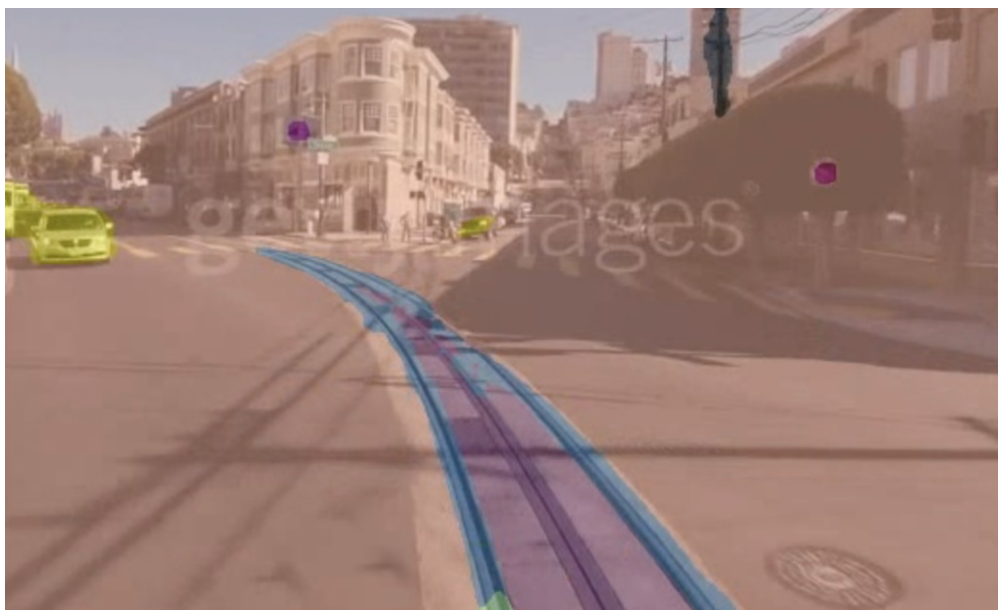


Рис. 16: Результаты сегментации SegFormer

Результат:

- Средняя скорость: 32 FPS
- Средняя точность: 64%



Рис. 17: Результаты сегментации DeepLabV3+

Результат:

- Средняя скорость: ≤ 1 FPS
- Средняя точность: 41%

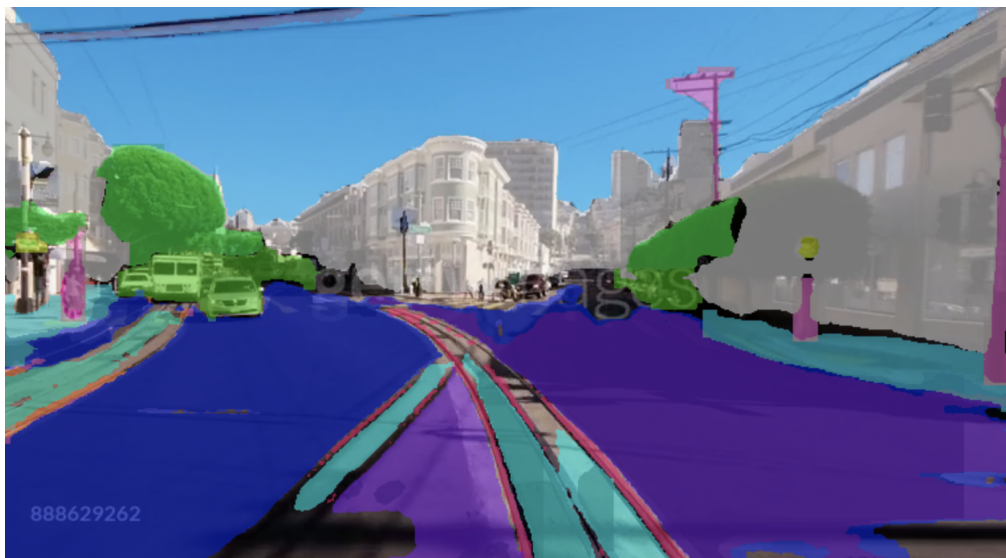


Рис. 18: Результаты сегментации YOLOv8n-seg

Результат:

- Средняя скорость: 34 FPS
- Средняя точность: 55%

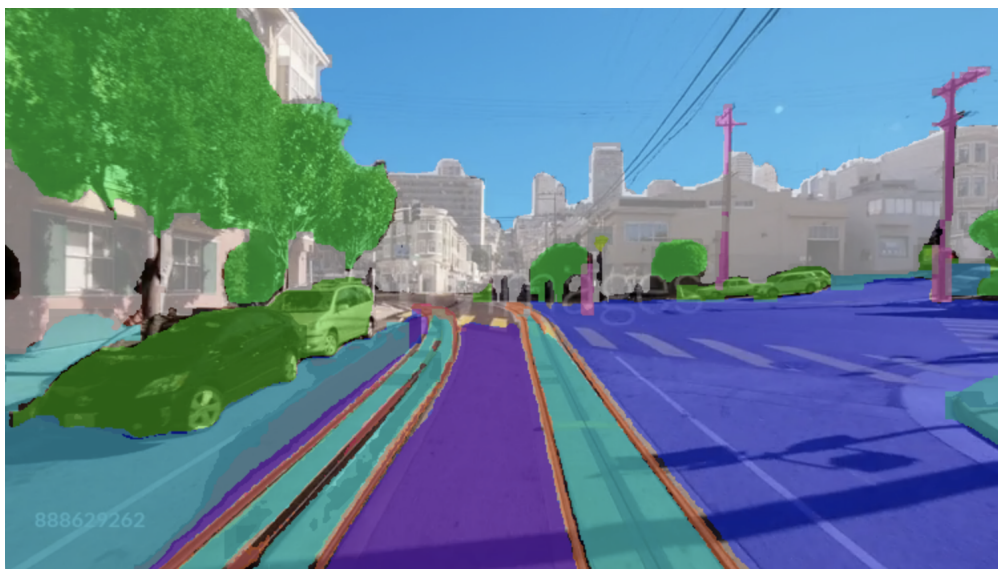


Рис. 19: Результаты сегментации YOLOv8s-seg

Результат:

- Средняя скорость: 30 FPS
- Средняя точность: 62%

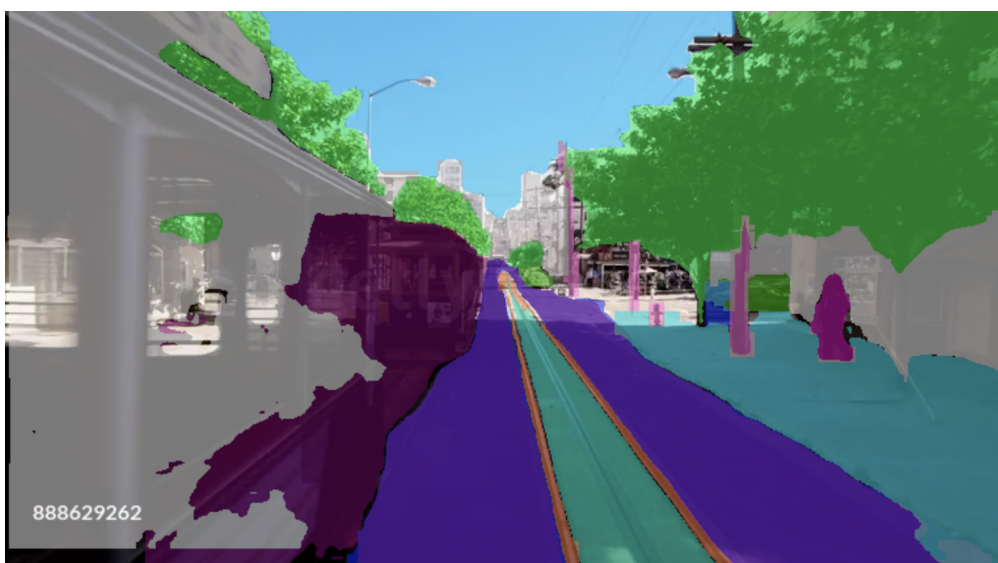


Рис. 20: Результаты сегментации YOLOv11s-seg

Результат:

- Средняя скорость: 31 FPS
- Средняя точность: 71%



Рис. 21: Результаты сегментации YOLO26s-seg

Результат:

- Средняя скорость: 32 FPS
- Средняя точность: 69%

Заключение

Выбранная модель YOLOv11s-seg, будет использоваться для дальнейшего поиска оптимальных гиперпараметров, с помощью библиотеки Optune, которая ищет оптимальные гиперпараметры более точно, но так же является более тяжелой для подбора.

Так же в последствии будет использоваться кросс-валидация с методом leave one out и пятью блоками (Fold), для более точной оценки модели.

Далее разрабатывается алгоритм выделения зоны интереса RoI, дорисовкой полигонов вокруг сегментированных рельс, и реализация инференса модели YOLOv8s для детекции посторонних объектов в режиме реального времени.

Список литературы

- [1] Скобцов Ю. А., Сперанский Д. В. Эволюционные вычисления: Учебное пособие. – М.: Национальный Открытый Университет «ИНТУИТ», 2012. – 331 с., ил. – (Серия «Основы информационных технологий»).
- [2] Алиев М. В., Бербенцев Д. А., Немыкин В. О., Алиева С. М. Алгоритмы распознавания объектов в видеопотоке и определение свойств их взаимного расположения // Вестник Адыгейского государственного университета. Сер. Естественно-математические и технические науки. – 2024. – № 2(341). – С. 27–34. DOI: 10.53598/2410-3225-2024-2-341-27-34.
- [3] Зотов С. С., Яковлев А. А., Колчищев Д. А. Обнаружение объектов в реальном времени с помощью алгоритмов распознавания YOLO // Синергия наук. – 2018. – № 23. – С. 1245–1261.
- [4] Чочиа П. А. Сегментация изображений на основе анализа расстояний в пространстве признаков // Информационные процессы. – 2020. – № 6. – С. 97–110.
- [5] Zhang H., Ullah N., Sharnah A. CA-SegFormer: An improved coordinate attention-enhanced model for crack segmentation in concrete structures // Case Studies in Construction Materials. – 2025. – Vol. 23.
- [6] Yanyan Z., Zhiliang S. Adioc loss: An Auxiliary descent IoC loss function // Engineering Applications of Artificial Intelligence. – 2022. – № 105453. – С. 117–133. DOI: <https://doi.org/10.1016/j.engappai.2022.105453>.
- [7] YOLOv8 Documentation [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolov8/> (дата обращения: 07.02.2026).
- [8] YOLOv5 Documentation [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolov5/> (дата обращения: 07.02.2026).
- [9] Xie E., Wang W., Yu Z., Anandkumar A., Alvarez J. M., Luo P. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers // arXiv [cs.CV]. – 2021. – 28 Oct.
- [10] RailSafeNet.train_SegFormer.py [Электронный ресурс] // GitHub. – Доступ: https://github.com/oValach/RailSafeNet/blob/main/train_SegFormer.py (дата обращения: 14.03.2026).
- [11] Мороз Е. С., Пикалёв Я. С. Анализ SOTA-архитектур для семантической аугментации // Информационные технологии и системы. – 2025. – № 4. – С. 39–60. DOI: 10.24412/2413-7383-2025-4-39-60-70.
- [12] Xie E., Wang W., Yu Z., Anandkumar A., Alvarez J. M., Luo P. Details of MiT Series // arXiv [cs.CV]. – 2021. – (Supplementary material).
- [13] DeepLabV3Plus-RailSem19 [Электронный ресурс] // GitHub. – Доступ: <https://github.com/pideyi1025/DeepLabV3Plus-RailSem19/tree/main> (дата обращения: 14.03.2026).
- [14] Springer Link. Статья по компьютерному зрению [Электронный ресурс] // Springer. – Доступ: <https://link.springer.com/article/10.1007/s11554-023-01380-x> (дата обращения: 14.03.2026).
- [15] Instance Segmentation. Val [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/tasks/segment/#val> (дата обращения: 14.03.2026).
- [16] YOLOv8. Key Features [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolov8/#key-features-of-yolov8> (дата обращения: 14.03.2026).
- [17] YOLOv11. Key Features [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolov11/#key-features-of-yolov11> (дата обращения: 14.03.2026).

- [18] YOLO26 Documentation [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/ru/models/yolo26/> (дата обращения: 14.03.2026).
- [19] Hyperparameter Tuning [Электронный ресурс] // Ultralytics. – Доступ: <https://docs.ultralytics.com/guides/hyperparameter-tuning/> (дата обращения: 14.03.2026).